TEXAS A&M UNIVERSITY | DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CASA: A Context-Aware Security Pipeline for Personal AI Agents

Chia-Chiao Liu | ccl@tamu.edu

TEXAS A&M UNIVERSITY

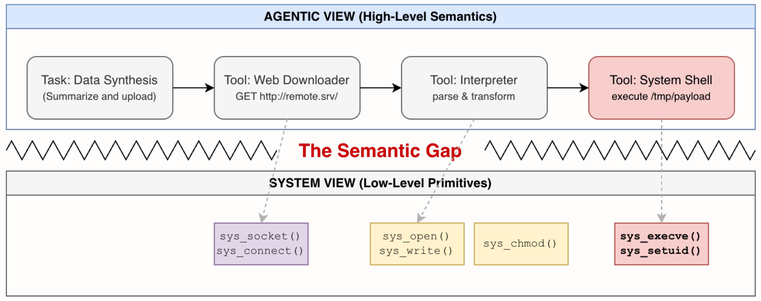
Motivation

Large language model agents increasingly act as autonomous software workers: they read files, invoke tools, open network connections, and execute commands on behalf of users.

This creates a new security problem: suspicious agent behavior is often **not visible in a single event**. It emerges from a sequence of ordinary-looking actions whose meaning only becomes clear over time, a **semantic gap** between low-level system observations and high-level agent intent.

CASA bridges this gap by treating a **session** (one end-to-end agent invocation) as the unit of analysis, reasoning over how actions compose across time rather than scoring each event in isolation.

Example Attack Pattern



The agent begins with a seemingly legitimate task: data synthesis and summarization. To complete it, the agent downloads external content, passes it into an interpreter, and executes a payload at `/tmp/payload`. At the agentic level, this is a coherent *download* → *process* → *execute* workflow.

From the OS perspective, however, the same workflow appears only as a fragmented sequence of individually benign syscalls: `sys_connect()`, `sys_open()`, `sys_write()`, `sys_execve()`, and others. No single event signals an attack.

Malicious intent only becomes apparent when the full execution chain is analyzed holistically, which is exactly what existing event-centric defenses cannot do.

Why Existing Defenses Fall Short

Existing runtime monitors provide strong low-level visibility, but remain **event-centric**: they observe individual syscalls without reconstructing how those events compose into agent behavior.

This leaves a gap between three levels of meaning:

- what the kernel sees — isolated, individually benign events
- what a human recognizes — a suspicious multi-step behavior
- what a policy must reason about — intent unfolding across time

Contribution

CASA contributes:

- a **two-plane architecture** separating eBPF-based event collection from user-space security analysis
- a **session model** for tracking multi-process agent activity
- a **structured context representation** over execution, capability, and behavioral history
- a **programmable scoring engine** that detects multi-step behaviors via concise, high-level rules
- a prototype implementation targeting **OpenClaw** (an open-source autonomous AI agent framework)

Threat Model

Our goal is to **detect and characterize** suspicious runtime behavior , not to prevent all possible attacks before execution.

We consider scenarios where an AI agent is indirectly influenced through prompt injection, malicious tool outputs, or unintended instruction execution. The attacker requires no direct system access. Instead, they steer the agent into system-level actions such as file access, process execution, and network communication.

In Scope

- multi-step behaviors that only become meaningful through cross-event correlation
- agent-induced system activity across processes and execution stages
- runtime detection and characterization of suspicious sessions

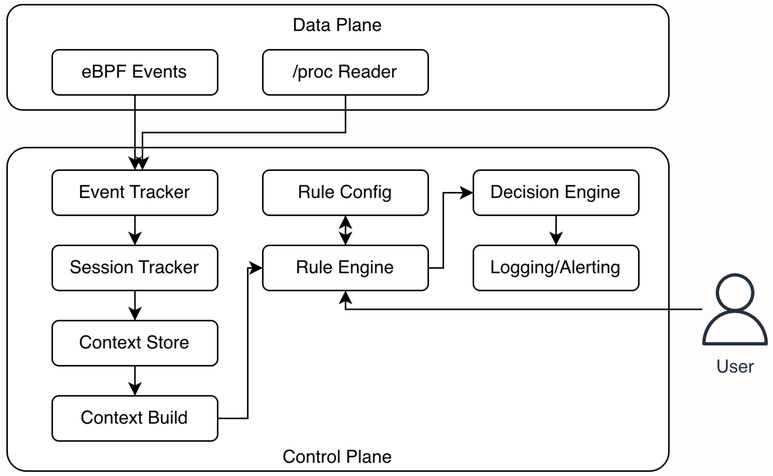
Out of Scope

- kernel compromise or attacks below the OS boundary
- guaranteed prevention before execution
- reasoning about model internals or prompt semantics in isolation

System Overview

CASA is organized as a **two-plane pipeline** that separates low-level event collection from higher-level security analysis.

- **Data Plane** sits at the OS boundary and observes what the agent actually does. eBPF probes capture process execution, file access, and network events in real time. A `/proc` reader supplements each event with process-state attributes such as Linux capabilities and seccomp mode.
- **Control Plane** turns raw events into security decisions. Incoming events are attributed to a session, used to incrementally build structured context, evaluated against a rule set, and scored. When the cumulative score crosses a threshold, the system emits an audit or alert record.



Session-Centered Analysis

The core analysis unit in CASA is a **session**, corresponding to one end-to-end agent invocation. Rather than scoring each system event independently, CASA maintains a running view of all activity within that invocation window, preserving continuity across:

- processes spawned by the agent and their children
- programs that replace themselves with another binary mid-run
- multi-step behaviors that span several tools or commands

This lets CASA reason not just about what happened, but about how it originated.

Agent-Aware Filtering

Before context is derived, CASA applies two layers of filtering to keep session context focused on agent-driven actions rather than ambient noise.

- **Provider filtering** is specific to AI agent deployments. CASA actively resolves the IP addresses used by LLM API providers at startup, including Cloudflare edge nodes, and excludes all matching traffic from security context. This prevents routine model communication from being misread as suspicious network activity.
- **Event filtering** removes low-level background noise that is unrelated to agent behavior: dynamic linker activity, common helper commands, and loopback traffic. This prevents benign system events from polluting session context and inflating risk scores.

Derived Context

For each session, CASA derives structured context along three dimensions that together describe what the agent did, what privileges it held, and how its behavior evolved over time.

- **Execution** captures the shape of the process tree: how deep the chain runs, whether a shell or network tool appeared, and whether anything was executed from a suspicious or non-persistent location.
- **Capability** records the privilege posture of processes in the session, including whether dangerous Linux capabilities were present and whether syscall filtering was disabled.
- **History** tracks cross-event patterns within the session, such as whether a network connection preceded a process execution, whether a sensitive file was accessed before data left the host, or whether something was written and then executed at the same path.

Configuration

Category	Examples
Behavior definitions	shell names, suspicious path patterns
Detection thresholds	burst windows, lineage depth
Network exclusions	LLM endpoints, known CIDRs
Rule parameters	weights, enable flags, score thresholds

Programmable Rule Engine

Context fields serve as the vocabulary for CASA's rule expressions. Rules evaluate over **derived context** rather than raw events directly. This allows multi-step behaviors to be expressed concisely without manual state tracking. Each rule triggers **at most once per session**, so cumulative scores reflect the breadth of suspicious behavior rather than the frequency of any single event.

Rules are loaded from a configuration file and can be **reloaded at runtime** without restarting the pipeline. On each reload, the engine validates rule syntax before applying changes, ensuring that misconfigured rules never silently affect scoring.

A representative rule:

```
history.connect_then_exec && execution.shell_in_chain
```

This captures a shell-mediated network-to-execution pattern. The total session risk is computed as the weighted sum of all triggered rules:

$$\text{Risk}(s) = \sum_{i=1}^{n} w_i \cdot f_i(s)$$

where w_i is the weight of rule i, and $f_i(s) \in \{0, 1\}$ indicates whether rule i triggered in session s.

Decision Outputs

CASA uses a two-threshold model to separate routine activity from noteworthy behavior.

Score	Action
$< T_1$	silent
$T_1 \leq \text{score} < T_2$	audit record emitted
$\geq T_2$	alert record emitted

Each audit and alert record includes the triggering event, the evaluated session context, the cumulative score, and the newly matched rules. This gives operators a lightweight explanation of *why* a session was flagged, not just *that* it was flagged.

Evaluation Summary

CASA is evaluated against a 30-case suite of 10 benign, 10 malicious, and 10 boundary workloads, each designed to exercise multi-step behaviors rather than isolated syscalls.

Category	Total	Correct	Note
Benign	10	9 silent	1 FP: boundary-like behavior
Malicious	10	9 alerted	1 FN: LLM did not realize behavior
Boundary	10	8 flagged	2 FP: reasonable given mixed signals

CASA reliably detects cross-step behaviors when realized at runtime, with low overhead and maintainable, human-readable rules.

Runtime Overhead

CASA is implemented in Go, with eBPF probes written in C and rules evaluated using the Common Expression Language (CEL). Go was chosen for its strong concurrency model and mature eBPF ecosystem support, while CEL provides a lightweight, sandboxed, and serializable rule language that can be evaluated safely without executing arbitrary code.

Overhead scales with event intensity rather than baseline computation, making CASA practical as an always-on monitor even under active workloads.

- CPU usage remains stable at roughly **1.2%**
- Memory footprint stays around **60 MB RSS**
- Internal event-to-log latency averages **123 µs**

Engineering Impact

CASA reduces the engineering cost of behavioral security by replacing low-level event-correlation logic with concise, high-level rules. The table below compares each 7-line CASA rule against the equivalent hand-written detection logic measured in lines of code.

Behavior	CASA Rule	Hand-written
Sensitive file → network	7 lines	1,353 LoC
Connect → exec in shell lineage	7 lines	1,602 LoC
Write → execute same path	7 lines	1,597 LoC

This compression is possible because CASA rules operate over derived context, offloading all event correlation, session tracking, and state management to the pipeline.

Future Work

Several directions remain open for future development:

- Evaluate across more agent runtimes and task types to improve robustness to variable LLM execution
- Reduce environment-specific noise through adaptive filtering calibrated to different deployment contexts
- Strengthen overhead evaluation with multi-run measurements and statistical analysis
- Extend context extraction and explore automated rule generation to reduce manual policy authoring